

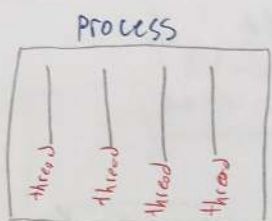

Process Management

Process:

A Process can be thought of as a Program in execution

thread:

A thread is the unit of execution within a Process. A Process can have anywhere from just one thread to many.



Program and Process
↓ ↓
class object

Process state:

Process state is defined by the current activity of it.

Process states:

New: the Process is being created.

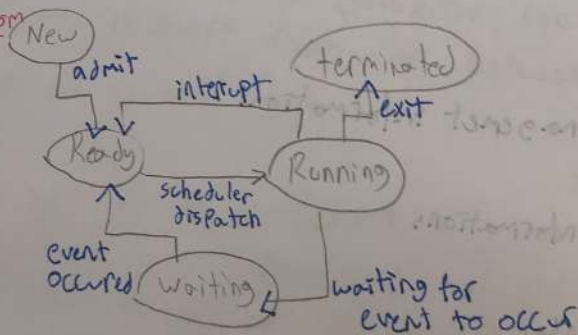
Running: Instructions are being executed.

waiting: The Process is waiting for some event to occur.
(I/O completion etc.)

Ready: The Process is waiting to be assigned to a processor.

Terminated: Process has finished execution.

Process state diagram



Process control blocks

Processes are represented as **PCB** (process control block) in the operating system.

Process state
Process number (id)
Program counter
Registers
Memory limits
list of open files

Process number (id): each Process's unique ID. So operating system can identify it.

Process state: represents the process state.

Program counter: indicates the address of the next instruction that has to be executed for that particular process.

CPU registers: the registers being used by the process.

CPU scheduling information: it knows the priority of processes and according to it it determines the processes will get executed. How much time it will take.

Memory management information: Memory being used by the process.

Accounting information: keeps a count of the used resources in the particular process.

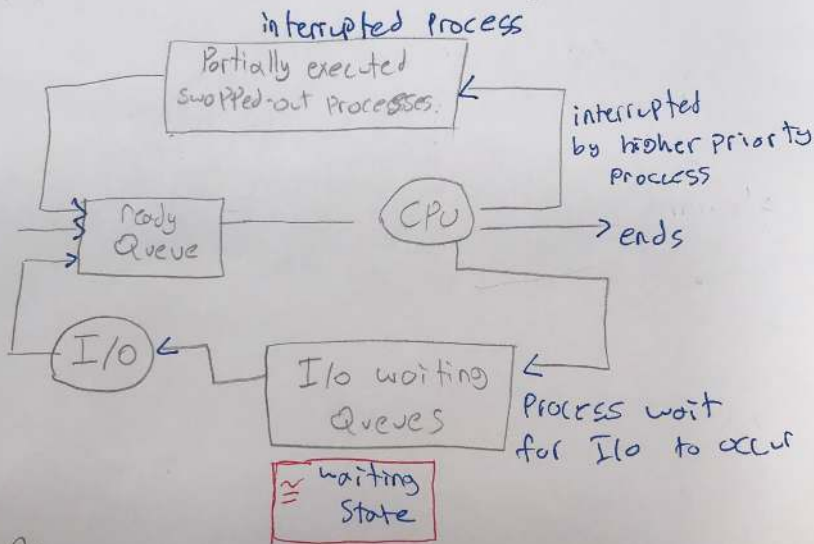
I/O status info: represents the I/O assigned to a process.

Process scheduling:

Selects a Process to be executed.

JOB QUEUE: the list of all the Processes in the system

READY QUEUE: the Process that are ready and waiting to be executed.



OS's file descriptor var 3 modlu

✳ when process creates new Process 2 scenarios are available

1- Parent continue executing concurrently with its children.

2- Parent waits until some or all children have terminated.

children can have some or all Parent resources.

Adress Space Possibilities

1- duplicate of Parent Process (same program and data as Parent)

2- child Process has new Program loaded to it.

Process termination

✳ A Process terminates when it finishes executing its final statement and asks the operating system to delete it by using `exit()` system call.

✳ Process may return status value to its Parent via `wait()` system call.

All resources of terminated Process are deallocated.

termination circumstances

Process can cause termination of another Process via system call.

Parent

InterProcess Communication

Processes running could be independent or cooperating

independent: cannot affect or be affected by other Processes.

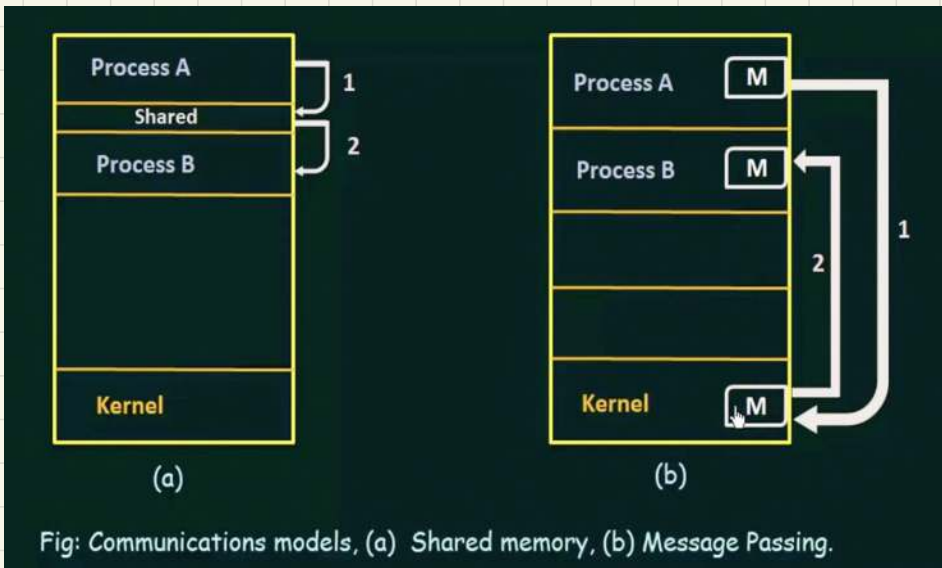
cooperating: they can affect or be affected.

* Any process that shares data is cooperating process

IPC: (inter process communication)

Processlerin haberleşmesi için 2 yöntem var.

- 1- Shared memory: they use same region of memory. Thus read/write will affect directly.
- 2- Message passing communication occur in form of messages between cooperating processes.



Shared memory:

- Communicating Processes needs to establish a region of shared memory.
- it resides in the address space of the process that creating shared-memory segment.
- other Processes have to attach that region to their own address space.

Message Passing Systems

- allow processes to communicate without sharing some address space.
- useful in distributed environments

2 functionalities:

1. send message
2. receive message

• messages could be fixed or variable size

fixed-size:

System-level implementation is easy. But it makes use of them harder (programming with them)

variable size:

system-level implementation harder. make use of them is easier

if processes P and Q wants to communicate using messages they have to establish a communication link.

exchanges messages via `send()` / `receive()` operations

Naming:

Processes that wants to communicate must have a way to refer to each other. they can use Direct or indirect communications.

Direct:

explicitly name recipient or sender of the communication.

`send(P, message)` send a message to Process P.
 ↓
 recipient

`receive(Q, message)` receive message from Process Q.
 ↓
 sender

Properties:

- link established with exactly 2 Processes
- each pair of processes have only 1 link.

Indirect Communications

messages are sent to and received from mailbox, or Ports

mailbox:

- messages are placed and removed.
- each mailbox has unique ID.
- 2 Process must have shared mailbox to communicate.
- $SEND(A, message)$ → send to mailbox A
 → mailbox identifier.
- $receive(A, message)$ → receive from mailbox A

A communication link in this scheme has the following properties:

- A link is established between a pair of processes only if both members of the pair have a shared mailbox.
- A link may be associated with more than two processes.
- Between each pair of communicating processes, there may be a number of different links, with each link corresponding to one mailbox.

Now suppose that processes P1, P2, and P3 all share mailbox A



Process P1 sends a message to A, while both P2 and P3 execute a receive() from A. Which process will receive the message sent by P1?

Çözüm:

- let 2 processes communicate at most.
- allow one process at a time to execute receive() operation
- allow system to choose who will receive, notify sender who received.

Synchronous or asynchronous communication:

• message passing:

- Blocking (synchronous)
- non blocking (asynchronous)

Blocking-send

sending process is blocked until message is received by receiving process or by mailbox

Non-blocking-send

sending process sends message and resume operation

Blocking-receive:

receiver blocks until a message is available

non-blocking receive

receive message or null.

Buffering:

Queue of messages attached to communication link.

3 types of buffer

- 1- Zero capacity: no messages are queued sender must wait for receiver.
- 2- Bounded capacity: finite length of n . if full sender must wait.
- 3- Unbounded capacity: infinite length sender never waits.

Sockets:

- socket is defined as an end point for communication
- A pair of processes communicating over a network employ a pair of sockets
 - one for each process.
- A socket is defined by an IP address concatenated with a Port number.

RPC (remote Procedure calls)

- communicating 2 processes on different systems.
- it similar to IPC mechanism -
- we need to use message-based scheme
- data have to be well structured as it will travel over the internet.
- when client invoke remote procedure the RPC will call the appropriate stub. and provide the params that were provided to the remote procedure. and the stub will locate Port on the server and marshall the parameters.
- marshalling parameters: Packaging the parameters into a form that can be transmitted over a network.

- the stub then transmit a message to the server using **message passing**
- similar stub on the server side receives this message and invokes the procedure on the server.

issues

- Differences in data representation on the client and server machine
- Big endian - little endian

local procedure calls fail only in extreme circumstances. **RPC** can fail, or be duplicated and executed more than once, as a result of common network errors

how does a client know the port numbers on the server?

$$\frac{1}{0.4 + \frac{0.6}{2}} = \frac{1}{0.4 + 0.3} = \frac{1}{0.7} = \frac{10}{7}$$

Solution

RPC systems define a machine independent representation of data. One such external data representation. (XDR)

using ACK make sure that RPC is sent exactly once. client keep sending until get ACK.

Port binding can be done by rendezvous mechanism. OS provides a rendezvous (or **matchmaker**) service to connect client and servers.

CPU Scheduling

its in the basis of multiprogrammed operating systems.

* Objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.

* In single processor computers when a process is waiting for I/O it will hold CPU and will waste time. So; with multiprogramming we try to use wasted time productively.

- Several Processes are kept in memory at one time.

* When a process has to wait, the operating system takes the CPU away from waiting process and gives it to another process.

* CPU Burst: Process is being executed in the CPU.

* I/O Burst: CPU is waiting for I/O for further execution. final CPU burst ends with system request to terminate execution.

↓
load store
add store
read from file } CPU burst

wait for I/O } I/O burst

store increment
index
write to file } CPU burst

wait for I/O } I/O burst

whose
Her. into

rose
fill

3.

it

tion. em

body
access

ested
erally

Queue
nd when

→ tim

$$\begin{array}{r} 0 \\ 24 \\ 27 \\ \hline (0+24+27) \\ 3 \end{array}$$

Preemptive and Non-Preemptive scheduling

* CPU scheduler: selects the next Process that will get executed by the CPU.

* Dispatcher: gives control of the CPU to the Process that was selected by the CPU scheduler.

Dispatch latency: the time it takes to stop one Process and start another running.

CPU Scheduling's 4 circumstances:

- 1- when a Process switches from running state to waiting state.
(wait for I/O)
 - 2- when a Process switches from the running state to the ready state (interrupt occurred)
 - 3- when a Process switches from waiting state to the ready state. (completion of I/O)
 - 4- terminates.
- in situations 1 and 4 the CPU doesn't take decision and it must select other Process, since either it's terminated or in the waiting state. (non-preemptive)
- in 2 and 3 when CPU is back shall it continue executing its last Process? or which decision it must take. (preemptive)
- non-preemptive: the CPU will never be taken out from Process until it finish execution or it goes to waiting state.
- Preemptive: CPU may be taken away while its executive.

Scheduling Criteria

- 1- CPU utilization
- 2- Throughput
- 3- turnaround time
- 4- waiting time
- 5- response time

1- CPU utilization:

we want to keep the CPU as busy as possible. theoretically from 0 to 100 percent. In practice 40% - 90%.

2- throughput:

the measure of processes gets completed per time unit.

3- turnaround time:

the time from submission of the process to its completion.

4- waiting time:

waiting time is the sum of the periods spent waiting in ready queue. CPU scheduling algorithm affects waiting time of a process.

5- response time

In an interactive system, turnaround time may not fit well. So response time is from the time submission is requested till it get first response. the turnaround time is generally limited by the speed of the output device.

Scheduling algorithms

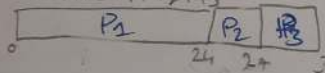
FCFS (First-come First serve):

- who ever come first takes CPU first. Uses FIFO Queue.
- when a process wants to execute it enters the queue and when it start running it gets out.

Process	Burst time
P ₁	24
P ₂	3
P ₃	3

Arrive time = 0

arrive P₁, P₂, P₃



waiting time

$$P_1 = 0$$

$$P_2 = 24$$

$$P_3 = 27$$

$$\text{avg} = \frac{(0 + 24 + 27)}{3} = 17$$

If Processes arrives P_2, P_3, P_1 :



waiting time

$$P_2 = 0$$

$$P_3 = 3$$

$$P_1 = 6$$

$$\text{avg} = \frac{0+3+6}{3} = 3$$

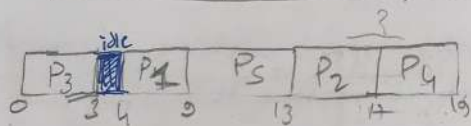
★ The FCFS scheduling algorithm is non preemptive.

convoy effect: if Process with higher burst times arrives before the Process with smaller burst time.

ex)

Process ID	Arrival time	Burst time
P_1	4	5
P_2	6	4
P_3	0	3
P_4	6	2
P_5	5	4

given table calculate average waiting time and average turnaround time. using **FCFS**.



Turn around = completion time - arrival time

waiting time = Turnaround time - Burst time

Process ID	completion time	turnaround	waiting time
P_1	9	$9-4=5$	$5-5=0$
P_2	11	$11-6=5$	$5-4=1$
P_3	3	$3-0=3$	$3-3=0$
P_4	13	$13-6=7$	$7-2=5$
P_5	7	$7-5=2$	$2-4=-2$

SJF (Shortest-job-first scheduling)

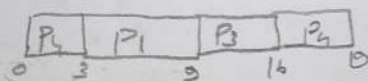
- when CPU is available it is assigned to the Process that has the smallest next CPU burst. if 2 Processes have same length FCFS is used.

SJF can be Preemptive or non-preemptive.

ex)

Process ID	Burst time
P ₁	6
P ₂	8
P ₃	7
P ₄	3

draw gantt chart using SJF (non-preemptive).



waiting time

$$P_4 = 0$$

$$P_1 = 3$$

$$P_3 = 9$$

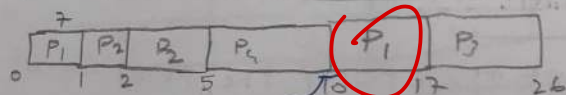
$$P_2 = 16$$

$$\text{avg} = 7$$

ex)

Process ID	Arrival time	Burst time
P ₁	0	8
P ₂	1	4
P ₃	2	3
P ₄	3	5

draw gantt and calculate average time using SJF (Preemptive).



$$\text{turn} = 17 - 0$$

$$17 - 1 \rightarrow$$

$$P_1: 7$$

$$P_2: 0$$

$$P_3: 5$$

waiting time = total waiting time - total executed - Arrival time.

$$P_1 = 17 - 1 - 0 = 9$$

$$P_2 = 1 - 0 - 1 = 0$$

$$P_3 = 17 - 0 - 2 = 15$$

$$P_4 = 5 - 3 = 2$$

$$\text{AVG} = 6.5$$

* Preemptive SJF also called shortest remaining time first scheduling
 * It is impossible to know the next CPU request of a process.

Q. Using Preemptive SJF calculate average waiting time

Process ID	Arrival time	Burst time
P ₁	0	12
P ₂	2	4
P ₃	3	6
P ₄	8	5

calculate average time?

P₁: 10
P₂: 20
P₃: 4
P₄: 5

P ₁	P ₂	P ₃	P ₄	P ₁
0	2	6	12	27

waiting time: total - burst - arr

$$P_1 = 17 - 12 - 0 = 5$$

$$P_2 = 2 - 0 - 2 = 0$$

$$P_3 = 6 - 0 - 3 = 3$$

$$P_4 = 12 - 0 - 8 = 4$$

$$22/4 = 5.5$$

Scheduling

Priority scheduling

- Each process has a priority. The process with the highest priority gets executed. If equal use FCFS.
 - Priority scheduling can be preemptive or non-preemptive.
- Preemptive priority scheduling:

The algorithm will preempt the CPU if the priority of the newly arrived process is higher than the running one.

Non-preemptive:

New process will have to wait the process to finish.

Problem: Starvation may occur. (small priority processes may never execute)

Solution: Aging: as time progresses increase the priority of process.

ex) Consider processes arrives at same time $t=0$. Small value of Priority means the higher. use priority scheduling.

Process ID	Burst time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

waiting time

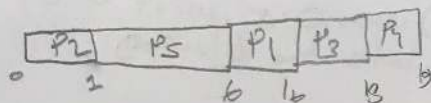
$$P_1 \sim 6$$

$$P_2 \sim 0$$

$$P_3 \sim 16 \quad \text{Avg} \sim \frac{41}{5} = 8.2$$

$$P_4 \sim 8$$

$$P_5 \sim 1$$



ex) P1 is highest priority. Calculate average time using Preemptive Priority scheduling algorithm.

Process ID	Arrival time	Burst time	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	4
P5	9	16	4

$$2P_1: 11$$

$$1P_4: 10$$

$$0P_2: 28$$

$$4P_5:$$

$$3P_3:$$

$$P_1 = 40 - 2 - 0 = 38$$

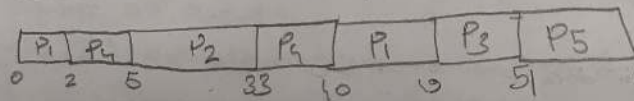
$$P_2 = 5 - 0 - 5 = 0$$

$$P_3 = 49 - 0 - 12 = 37$$

$$P_4 = 33 - 3 - 2 = 28$$

$$P_5 = 51 - 0 - 9 = 42$$

$$60 + 75 = 135 \div 5 = 27$$



waiting time = total - arrive - burst

Round - Robin algorithm

⑧

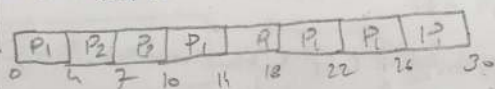
- * Round-robin scheduling algorithm is designed especially for time sharing systems.
 - * Each process gets a small unit of CPU time (time quantum). then the process is added to the end of the ready queue.
 - * Ready queue is treated as a circular queue.
- PS: if we choose a large time quantum RR will be FCFS.

ex)

arrive time 0, time quantum = 4. calculate the following:

Process ID	Burst time
P1	24
P2	3
P3	3

Gantt chart:



turnaround: completion - arrival

waiting: turn around time - Burst time

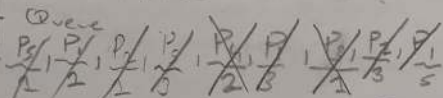
Process ID	Completion time	turnaround time	waiting time
P1	20	20 - 24 = -4	20 - 24 = -4
P2	7	7	7 - 3 = 4
P3	10	10	10 - 3 = 7

Avg wait = $\frac{17}{3} = 5.67$

method 2,

a) RR time quantum = 2, calculate average turnaround and waiting time.

Process ID	Arrive time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	2



Multilevel Queue.

* Ready Queue is Partitioned into separate Queues.

- foreground (interactive) — They have

- background (Batch)

- Different response time.

- Different scheduling needs.

* Process assigned permanently to a queue.

* Foreground Processes may have higher priority than background.

Important: each queue has different algorithm.

• Foreground - RR

• Background - FCFS

* Scheduling of Queues. Fixed Priority Scheduling. (serve all from foreground then from background). Possibility of starvation.

* Time-slice: each queue gets a certain amount of CPU time which it can schedule among its processes.

example of priority Process Queue:

